Thesis: A Comparative Study of Large Language Models for Financial Sentiment Analysis and Their Predictive Potential for Short-Term Stock Price Movement

Component: Experiment 2 - Model C: Mistral (7B Instruct v0.3)

Description: This notebook loads the prepared 1000 headline dataset and evaluates Mistral 7B on Experiment 2 using zero-shot prompting. It maps sentiment predictions to directional signals and computes directional accuracy, precision, recall, and F1-score based on next-day stock price movement.

Select T4 GPU as runtime.

```
# Install bitsandbytes — restart required after this

!pip install -q -U bitsandbytes accelerate
```

Go to runtime -> restart this session again -> then run cell 1 and run cell 2

```
# Import required libraries — Experiment 2c: Mistral

import pandas as pd
import numpy as np
import torch
import warnings
import gc

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)

warnings.filterwarnings("ignore")

print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available  : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU detected    : {torch.cuda.get_device_name(0)}")
```

```
PyTorch version : 2.10.0+cu128
CUDA available  : True
GPU detected    : Tesla T4
```

Add the API Key from Hugging Face using "Add New Secret" in Google Colab

```
# Connect to Hugging Face and load Experiment 2 dataset

from google.colab import userdata, drive
from huggingface_hub import login

drive.mount("/content/drive")

hf_token = userdata.get("HF_TOKEN")
login(token=hf_token)

df_exp2 = pd.read_csv("/content/drive/MyDrive/Thesis_Data/exp2_dataset.csv")

print("Dataset loaded.")
print(f"Total headlines : {len(df_exp2)}")
print(f"\nMovement distribution:")
print(df_exp2["movement"].value_counts())
```

```
Mounted at /content/drive
Dataset loaded.
Total headlines : 1000

Movement distribution:
movement
1    508
0    492
Name: count, dtype: int64
```

```
# Load Mistral 7B Instruct v0.3

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
```

```
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)

print("Loading Mistral 7B Instruct v0.3...")

mistral_tokenizer = AutoTokenizer.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.3",
    token=hf_token
)

mistral_model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.3",
    quantization_config=bnb_config,
    device_map="auto",
    token=hf_token
)

print("Mistral 7B loaded successfully.")
```

```
Loading Mistral 7B Instruct v0.3...
config.json: 100%                                                 601/601 [00:00<00:00, 33.6kB/s]
tokenizer_config.json:        141k/? [00:00<00:00, 2.68MB/s]
tokenizer.json:        1.96M/? [00:00<00:00, 22.1MB/s]
tokenizer.model: 100%                                             587k/587k [00:01<00:00, 2.92MB/s]
special_tokens_map.json: 100%                                     414/414 [00:00<00:00, 7.40kB/s]
model.safetensors.index.json:        23.9k/? [00:00<00:00, 1.23MB/s]
Download complete: 100%        14.5G/14.5G [05:14<00:00, 72.2MB/s]
Fetching 3 files: 100%                                            3/3 [05:13<00:00, 131.98s/it]
Loading weights: 100%                                             291/291 [00:59<00:00, 148.16it/s, Materializing param=model.norm.weight]
generation_config.json: 100%                                      116/116 [00:00<00:00, 12.1kB/s]
Mistral 7B loaded successfully.
```

```
# Define Mistral classifier — positive or negative only

import logging
logging.getLogger("transformers").setLevel(logging.ERROR)

def classify_mistral(text):
    prompt = f"""You are a financial sentiment classifier.

Classify the sentiment of this financial text as exactly one word: positive or negative.

Text: {text}

Respond with only one word: positive or negative.

Sentiment:"""

    inputs = mistral_tokenizer(
        prompt,
        return_tensors="pt"
    ).to(mistral_model.device)

    with torch.no_grad():
        outputs = mistral_model.generate(
            **inputs,
            max_new_tokens=5,
            do_sample=False,
            pad_token_id=mistral_tokenizer.eos_token_id
        )

    input_length = inputs["input_ids"].shape[1]
    generated = mistral_tokenizer.decode(
        outputs[0][input_length:],
        skip_special_tokens=True
    ).strip().lower()

    if "negative" in generated:
        return "negative"
    else:
        return "positive"

print("Mistral classifier ready.")
```

```
Mistral classifier ready.
```

```
# Run Mistral on 1,000 headlines dataset

headlines = df_exp2["headline"].tolist()
mistral_preds = []
total = len(headlines)

print(f"Running Mistral on {total} headlines...")
print("-" * 40)

for i, text in enumerate(headlines):
    label = classify_mistral(text)
    mistral_preds.append(label)

    if (i + 1) % 100 == 0:
        print(f"  Progress: {i+1}/{total}")

print(f"\nDone. Total predictions: {len(mistral_preds)}")
print(f"\nSentiment distribution:")
print(pd.Series(mistral_preds).value_counts())
```

```
Running Mistral on 1000 headlines...
----------------------------------------
  Progress: 100/1000
  Progress: 200/1000
  Progress: 300/1000
  Progress: 400/1000
  Progress: 500/1000
  Progress: 600/1000
  Progress: 700/1000
  Progress: 800/1000
  Progress: 900/1000
  Progress: 1000/1000

Done. Total predictions: 1000

Sentiment distribution:
positive    643
negative    357
Name: count, dtype: int64
```

```
# Map sentiment to directional prediction and evaluate

df_exp2["mistral_sentiment"] = mistral_preds
df_exp2["mistral_direction"] = df_exp2["mistral_sentiment"].map({
    "positive": 1,
    "negative": 0
})

true_movement = df_exp2["movement"].tolist()
pred_movement = df_exp2["mistral_direction"].tolist()

acc  = accuracy_score(true_movement, pred_movement)
prec = precision_score(true_movement, pred_movement, average="macro")
rec  = recall_score(true_movement, pred_movement, average="macro")
f1   = f1_score(true_movement, pred_movement, average="macro")

print("=" * 50)
print("Mistral 7B — Experiment 2 Results (next-day movement)")
print("=" * 50)
print(f"  Directional Accuracy : {acc:.4f}  ({acc*100:.2f}%)")
print(f"  Precision            : {prec:.4f}")
print(f"  Recall               : {rec:.4f}")
print(f"  F1-Score             : {f1:.4f}")
print("=" * 50)
print(classification_report(true_movement, pred_movement,
      target_names=["Down (0)", "Up (1)"]))
```

```
==================================================
Mistral 7B — Experiment 2 Results (next-day movement)
==================================================
  Directional Accuracy : 0.4870  (48.70%)
  Precision            : 0.4834
  Recall               : 0.4847
  F1-Score             : 0.4750
==================================================
              precision    recall  f1-score   support

    Down (0)       0.47      0.34      0.40       492
      Up (1)       0.50      0.63      0.55       508

    accuracy                           0.49      1000
   macro avg       0.48      0.48      0.48      1000
```

```
weighted avg       0.48      0.49      0.48      1000
```

```
# Save Mistral Experiment 2 results

mistral_exp2_scores = {
    "model": "Mistral 7B",
    "directional_accuracy": round(acc, 4),
    "precision": round(prec, 4),
    "recall": round(rec, 4),
    "f1_score": round(f1, 4)
}

print("Mistral 7B — Experiment 2 Results Summary")
print(pd.DataFrame([mistral_exp2_scores]))

df_exp2.to_csv("/content/drive/MyDrive/Thesis_Data/exp2_mistral_preds.csv", index=False)
print("\nSaved to Google Drive.")
```

```
Mistral 7B — Experiment 2 Results Summary
        model  directional_accuracy  precision  recall  f1_score
0  Mistral 7B                 0.487     0.4834  0.4847     0.475

Saved to Google Drive.
```